

Local vs Online: Complete Infrastructure & Deployment Guide

How to Run Everything on Your Local Machine, Switch Between Hosts, and Understand Every Component

Contents

Switching Between Local and Railway

- [The Big Picture — What We Changed and Why](#)
- [File-by-File: Every Change Explained](#)
- [The 5-Step Localhost Checklist](#)
- [Step 1: Stop Railway and Prepare Local Services](#)
- [Step 2: Your .env File — the Master Switch](#)
- [Step 3: Start Local PostgreSQL](#)
- [Step 4: Start Local Keycloak](#)
- [Step 5: Start Django and Verify](#)
- [Running setup_demo_data Locally](#)
- [Presentation Day — Two Scenarios](#)
- [Quick Reference: Local vs Railway Comparison](#)
- [Troubleshooting Localhost Issues](#)

Database Access & Understanding

- [Database Access — Viewing Your Data](#)
- [Users, Roles, and PendingAccess](#)

Hosting, Domains & Advanced Concepts

- [Changing Domain or Host Provider](#)
- [Advanced Concepts for Deep Understanding](#)

The Big Picture — What We Changed and Why

Before we changed anything for Railway, your project ran entirely on your local machine. You had:

```
Your Laptop:
├─ PostgreSQL (Windows service)  ← port 5432
├─ Django (runserver)             ← port 8000
├─ Keycloak (kc.bat start-dev)    ← port 8180
    └─ Keycloak users: admin / admin123
        └─ Realm: govasset
            └─ OIDC clients: govasset-django
                └─ Users: superadmin, moh_admin, etc.
```

To deploy to Railway (online), we made changes in two categories:

Category	What changed	Why
Code changes	settings.py, views.py, setup_demo_data.py	Make the code work in both environments
Environment changes	New env vars on Railway	Configure Django/Keycloak for Railway's infrastructure

The key insight: The code changes are **permanent** — they make the project work in BOTH local and online modes. You do NOT undo them. Only the environment (your `.env` file vs Railway's Variables tab) determines which mode you're in.

File-by-File: Every Change Explained

1. `config/settings.py` — The Core Configuration

Change A: `ALLOWED_HOSTS`

Before	After
<code>ALLOWED_HOSTS = ['localhost', '127.0.0.1', '0.0.0.0']</code>	<code>ALLOWED_HOSTS = config('ALLOWED_HOSTS').split(',')</code>

What it does: Django rejects requests that don't match this list. On Railway, the hostname is `government-assets-platform-production.up.railway.app`. We can't hardcode that in the file because it changes per deployment. So we moved it to an environment variable.

How to switch locally: In your `.env` file, add:

```
ALLOWED_HOSTS=localhost,127.0.0.1,0.0.0.0
```

Or leave it unset and the code will work because... wait, actually the code reads from env and splits on comma. If the env var is not set, it will fail. Let me check...

Actually, looking at the code, there's no default for `ALLOWED_HOSTS`. I need to add a default. Let me check.

Actually, earlier when I looked at the code:

```
ALLOWED_HOSTS = config('ALLOWED_HOSTS').split(',')
```

This uses `python-decouple`'s `config()`. If `ALLOWED_HOSTS` is not set in `.env`, it will raise an error. So for local development, the user needs to set it in `.env`:

```
ALLOWED_HOSTS=localhost,127.0.0.1,0.0.0.0
```

Change B: DATABASE_URL Support

What was added:

```
DATABASE_URL = config('DATABASE_URL', default=None)
if DATABASE_URL:
    parsed = urlparse(DATABASE_URL)
    DATABASES = {
        'default': {
            'ENGINE': 'django_tenants.postgresql_backend',
            'NAME': parsed.path.lstrip('/'),
            'USER': parsed.username,
            'PASSWORD': parsed.password,
            'HOST': parsed.hostname,
            'PORT': parsed.port,
        }
    }
else:
    DATABASES = {
        'default': {
            'ENGINE': 'django_tenants.postgresql_backend',
            'NAME': config('DB_NAME'),
            'USER': config('DB_USER'),
            'PASSWORD': config('DB_PASSWORD'),
            'HOST': config('DB_HOST'),
            'PORT': config('DB_PORT', cast=int),
        }
    }
```

What it does: Railway gives us a `DATABASE_URL` like `postgresql://postgres:abc123@host:5432/railway`. The code detects it, parses it, and builds the database config automatically. Locally, `DATABASE_URL` is not set, so it falls back to the old method using individual `DB_NAME`, `DB_USER`, etc. from `.env`.

How to switch locally: Do NOT set `DATABASE_URL` in your `.env`. Keep using `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT` as before. The `else` branch handles this.

Your `.env` should have:

```
DB_NAME=gov_asset_platform
DB_USER=postgres
DB_PASSWORD=your_local_postgres_password
DB_HOST=localhost
DB_PORT=5432
```

Change C: SECURE_PROXY_SSL_HEADER

What was added:

```
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
```

What it does: Railway terminates HTTPS at their proxy and forwards HTTP to Django internally. Django needs this setting to know the original request was HTTPS. Without it, Django generates `http://` redirect URLs instead of `https://`, breaking SSO and security.

How to switch locally: This is **harmless locally** — the header is never sent by your local browser, so Django simply ignores it. You do NOT need to remove it. But if you want to be safe:

Add to your `.env`:

```
SECURE_PROXY_SSL_HEADER=False
```

And wrap it in code. But actually, it's fine as-is. Just leave it.

Change D: PLATFORM_BASE_URL

What was added:

```
PLATFORM_BASE_URL = config('PLATFORM_BASE_URL', default='http://localhost:8000')
```

What it does: Used by views (like logout) that need to know the full URL of the Django server. Defaults to `http://localhost:8000` (your local dev URL).

How to switch locally: You don't need to do anything. The default is `http://localhost:8000`, which is exactly what localhost uses. On Railway, we set it to `https://government-assets-platform-production.up.railway.app`.

Change E: Whitenoise Middleware

What was added to MIDDLEWARE:

```
'whitenoise.middleware.WhiteNoiseMiddleware',
```

Right after `SecurityMiddleware`.

What it does: In production, Django does NOT serve static files (CSS, JS, images). Normally you need a web server like Nginx for that. Whitenoise lets Django serve its own static files in production, removing the need for Nginx.

How to switch locally: Harmless locally. In fact, `runserver` already serves static files without whitenoise (it does it automatically). Whitenoise just doesn't interfere. Leave it.

Change F: Requirements.txt

Added	Why
<code>gunicorn</code>	The WSGI server that runs Django in production (Railway uses it). Locally, you still use <code>runserver</code> .
<code>whitenoise</code>	Serves static files in production without Nginx

How to switch locally: These are already installed in your virtual environment. They don't affect `runserver`. Just leave them.

2. `authentication/views.py` — The Logout Fix

Before	After
<code>redirect('http://localhost:8000/login/')</code>	<code>redirect(platform_base + '/login/')</code>

What it does: When a user logs out, Django needs to redirect them back to the login page. Previously, this was HARDCODED to `http://localhost:8000` — which worked locally but broke on Railway (wrong URL, no HTTPS). Now it reads from `settings.PLATFORM_BASE_URL`, which adapts to whichever environment you're in.

How to switch locally: Nothing to do. `PLATFORM_BASE_URL` defaults to `http://localhost:8000`, so locally it works exactly as before.

3. `authentication/management/commands/setup_demo_data.py` — Fresh DB Fix

What was added:

```
from django.core.management import call_command

def _seed_ministries(self):
    # Create Ministry objects (creates PostgreSQL schemas)
    ministries = [
        {'name': 'Ministry of Health', 'schema_name': 'moh_schema'},
        {'name': 'Ministry of Finance', 'schema_name': 'mof_schema'},
    ]
    for m in ministries:
        ministry = Ministry.objects.create(...)
        ministry.save() # Creates the PostgreSQL schema

def _run_tenant_migrations(self):
    call_command('migrate_schemas') # Runs migrations inside tenant schemas
```

What it does: When you run `setup_demo_data` on an empty database (like a fresh Railway PostgreSQL), it creates the ministries AND runs the tenant migrations. Before this fix, the command would crash because it tried to create users in schemas that didn't exist yet.

How to switch locally: This runs in BOTH environments. Whether you're setting up a fresh Railway DB or a fresh local DB, `setup_demo_data` now works correctly. No changes needed.

The 5-Step Localhost Checklist

If you want to switch from Railway back to localhost:

- Step 1: Stop Railway services (optional – you can keep them running)
- Step 2: Update your `.env` file with local values
- Step 3: Start PostgreSQL (Windows service)
- Step 4: Start Keycloak (`kc.bat`)
- Step 5: Start Django (`runserver`) and verify

Step 1: Stop Railway and Prepare Local Services

Option A: Keep Railway running (recommended for testing)

You can keep Railway running while you work locally. They use different databases — Railway's PostgreSQL is separate from your local PostgreSQL. Changes on local don't affect Railway.

Option B: Stop Railway services

If you want to avoid confusion or save Railway resources:

1. Open <https://railway.app/dashboard>
2. Click each service (Django, PostgreSQL, Keycloak)
3. Click the three dots (...) → Stop or Delete

WARNING: If you **delete** Keycloak, the realm and users are gone permanently (PostgreSQL will keep them, but the new Keycloak instance won't connect to Railway PostgreSQL since you're going local anyway).

What to do with your local database

Before switching, you need to decide:

Situation	What to do
Your local PostgreSQL has the <code>gov_asset_platform</code> database with migrations	Nothing — it's ready
Your local PostgreSQL is fresh/empty	Create the database, run migrations, run <code>setup_demo_data</code>
You want to keep local data but also have Railway data	They are separate databases — no conflict

Step 2: Your `.env` File — the Master Switch

This is the **most important step**. Your `.env` file controls whether Django acts like localhost or Railway.

Open your `.env` file

Location: `D:\government_asset_platform\.env`

Make sure it has these values:

```
# — SECURITY —
SECRET_KEY=django-insecure-... (your existing key – don't change)
DEBUG=True # True for local, False for Railway

# — DATABASE (Local PostgreSQL) —
DB_NAME=gov_asset_platform
DB_USER=postgres
DB_PASSWORD=your_local_password
DB_HOST=localhost
DB_PORT=5432

# — KEYCLOAK (Local) —
KEYCLOAK_SERVER_URL=http://localhost:8180
KEYCLOAK_REALM=govasset
KEYCLOAK_CLIENT_ID=govasset-django
KEYCLOAK_CLIENT_SECRET=i9bDUIzrXNATomD5IAtxuowZDmsHKqfb
KEYCLOAK_ADMIN_USERNAME=superadmin
KEYCLOAK_ADMIN_PASSWORD=Admin@123

# — HOSTING —
ALLOWED_HOSTS=localhost,127.0.0.1,0.0.0.0

# — PLATFORM URL (used by logout, emails, etc.) —
PLATFORM_BASE_URL=http://localhost:8000

# — DO NOT SET THESE FOR LOCAL —
# DATABASE_URL= ← Leave this UNSET (no line at all)
# Do NOT add Railway-specific variables here
```

Critical: What to REMOVE from .env

If you added any of these to your `.env` during our Railway work, **delete them**:

```
# DELETE these lines if they exist:
# DATABASE_URL=postgresql://...
# CSRF_TRUSTED_ORIGINS=.up.railway.app
```

Why each value is what it is

Variable	Local value	Why
DEBUG=True	True	Shows debug info on errors — essential for development
DB_HOST=localhost	localhost	Your PostgreSQL runs on your machine

Variable	Local value	Why
<code>KEYCLOAK_SERVER_URL=http://localhost:8180</code>	<code>http://localhost:8180</code>	Your local Keycloak runs on port 8180
<code>ALLOWED_HOSTS=localhost,127.0.0.1,0.0.0.0</code>	Local addresses	Only accept connections from your machine
<code>PLATFORM_BASE_URL=http://localhost:8000</code>	<code>http://localhost:8000</code>	Django is served on port 8000

Step 3: Start Local PostgreSQL

Check if PostgreSQL is running

Method 1 — Windows Services:

1. Press Win + R, type: `services.msc`
2. Find: `postgresql-x64-17` (or whatever version you installed)
3. Status should say: Running

Method 2 — PowerShell:

```
Get-Service postgresql*
```

If PostgreSQL is not running

```
# Start it (replace version with yours)
net start postgresql-x64-17
```

If you need to create a fresh database

```
# Connect to PostgreSQL
psql -U postgres

# Create the database (if it doesn't exist)
CREATE DATABASE gov_asset_platform;

# Verify it was created
\l

# Exit
\q
```

Verify your database exists

```
psql -U postgres -d gov_asset_platform -c "\dn"
```

You should see schemas if you already ran setup_demo_data before.

Step 4: Start Local Keycloak

Find where Keycloak is installed

Look in `C:\keycloak-25.0.1\bin` (or whatever version folder you have).

Start Keycloak in development mode

```
cd C:\keycloak-25.0.1\bin
.\kc.bat start-dev --http-port=8180
```

Wait for this output (takes about 30 seconds):

```
2026-XX-XX XX:XX:XX,XXX INFO [io.quarkus] (main) Keycloak 25.0.1 on JVM (powered by
Quarkus) started in X.XXXs
```

Verify Keycloak is running

Open your browser: <http://localhost:8180>

You should see the Keycloak login page. Log in with: `superadmin / Admin@123`

If you see a blank page or error

```
Problem: Port 8180 is already in use
Fix: netstat -ano | findstr :8180
    Find the PID and: taskkill /PID <number> /F
    Then retry starting Keycloak

Problem: Keycloak was not fully installed
Fix: Download from https://www.keycloak.org/downloads
    Extract to C:\keycloak-25.0.1
    It should work immediately – no installation needed
```

Verify your Keycloak has the govasset realm and users

1. Open <http://localhost:8180/admin>
 2. Login: `superadmin / Admin@123`
 3. If you see `govasset` in the realm dropdown → you're good
 4. If you only see `master` realm → you need to recreate the govasset realm (see Troubleshooting below)
-

Step 5: Start Django and Verify

Activate your virtual environment

```
cd D:\government_asset_platform
.\venv\Scripts\Activate.ps1
```

Run migrations (if fresh database)

```
python manage.py migrate_schemas
python manage.py setup_demo_data
```

Start Django

```
python manage.py runserver 0.0.0.0:8000
```

Verify everything works

Test 1 — Django is running:

```
Open: http://localhost:8000
You should see the login page (not an error)
```

Test 2 — Log in via SSO:

```
Click "Login" or go to: http://localhost:8000/login/
You should be redirected to: http://localhost:8180/realms/govasset/...
Log in with: moh_admin / Admin@123
You should be redirected back to Django's dashboard
```

Test 3 — API works:

```
curl -X POST http://localhost:8000/api/auth/login/ `
-H "Content-Type: application/json" `
-d '{"username":"moh_admin","password":"Admin@123"}'
```

You should get a JSON response with tokens and user info.

Running setup_demo_data Locally

When to run it

Situation	Run setup_demo_data?
Fresh database, never used before	✅ Yes — required
Switching from Railway back to local and local DB is empty	✅ Yes — required
You already had a local database with data	❌ No — your data is there
You deleted all data and want to reset	✅ Yes — it's idempotent

What setup_demo_data does

```

setup_demo_data:
├─ Cleans old data (deletes all users, ministries)
├─ Creates _seed_ministries():
│   ├── Ministry of Health (moh_schema)
│   └── Ministry of Finance (mof_schema)
├─ Runs _run_tenant_migrations():
│   └── Creates all tables inside moh_schema and mof_schema
└─ Creates 6 demo users:
    ├── superadmin → SUPER_ADMIN → All ministries
    ├── moh_admin → MINISTRY_ADMIN → Ministry of Health
    ├── mnh_manager → AGENCY_MANAGER → Ministry of Health
    ├── rad_clerk → FACILITY_CLERK → Ministry of Health
    ├── moh_auditor → AUDITOR → Ministry of Health
    └── mof_admin → MINISTRY_ADMIN → Ministry of Finance
    (All passwords: Admin@123)

```

Important Note: `setup_demo_data` creates users in Django's database. It does NOT create them in Keycloak. You need to create Keycloak users separately (manually via admin console).

How to run it

```

cd D:\government_asset_platform
.\venv\Scripts\Activate.ps1
python manage.py setup_demo_data

```

Expected output:

```

Setting up demo data...
Seeding ministries...
Running tenant migrations...
Operations to perform...
  Synchronize unmigrated apps...
  ...
Creating demo users...
  Created superadmin (SUPER_ADMIN)
  Created moh_admin (MINISTRY_ADMIN)
  Created mnh_manager (AGENCY_MANAGER)
  Created rad_clerk (FACILITY_CLERK)

```

```
Created moh_auditor (AUDITOR)
Created mof_admin (MINISTRY_ADMIN)
Setup complete.
```

Presentation Day — Two Scenarios

Scenario A: Presenting Locally (your laptop)

Before presentation (10 minutes of setup):

```
# 1. Start PostgreSQL (if not running)
net start postgresql-x64-17

# 2. Start Keycloak (new terminal)
cd C:\keycloak-25.0.1\bin
.\kc.bat start-dev --http-port=8180
# Wait for "started in X.XXXs" message

# 3. Start Django (new terminal)
cd D:\government_asset_platform
.\venv\Scripts\Activate.ps1
python manage.py runserver 0.0.0.0:8000

# 4. Verify all three are running
# PostgreSQL: Windows Services → "Running"
# Keycloak:    http://localhost:8180 → login page
# Django:      http://localhost:8000 → login page
```

If the local database is empty (new laptop):

```
python manage.py migrate_schemas
python manage.py setup_demo_data
```

Then create users in local Keycloak:

1. Open <http://localhost:8180/admin>
2. Switch to **govasset** realm
3. Create 6 users (same usernames, passwords) matching the demo data
 - superadmin / Admin@123
 - moh_admin / Admin@123
 - mnh_manager / Admin@123
 - rad_clerk / Admin@123
 - moh_auditor / Admin@123
 - mof_admin / Admin@123

During presentation:

- Everything runs on YOUR laptop

- No internet required (except maybe for browser)
- URLs: <http://localhost:8000> (Django), <http://localhost:8180> (Keycloak)

Pros:

- No dependency on internet
- Faster (no network latency)
- Full control — nothing can break externally

Cons:

- Laptop performance matters (need PostgreSQL + Keycloak + Django running)
- If laptop crashes, presentation is over
- Screen might be small

Scenario B: Presenting via Railway (online)

Before presentation (check everything is running):

```
# 1. Check Railway status
# Open: https://railway.app/dashboard
# All 3 services should show: Deployed / Running

# 2. Test Django
# Open: https://government-assets-platform-production.up.railway.app
# Should show login page

# 3. Test Keycloak
# Open: https://keycloak-production-4f96.up.railway.app
# Should show login page

# 4. Test admin
# Open: https://keycloak-production-4f96.up.railway.app/admin
# Login: superadmin / Admin@123
```

If a service is down:

```
Open Railway dashboard → Click the service → Deployments → View logs
Look for errors. Common fixes:
- Out of memory → Check logs for "OutOfMemoryError"
- PostgreSQL → Check if it's running (services → PostgreSQL)
- Keycloak → Maybe need to recreate realm/users if container restarted
```

During presentation:

- Everything runs on Railway's servers
- Internet required
- URLs: <https://government-assets-platform-production.up.railway.app> (Django), <https://keycloak-production-4f96.up.railway.app> (Keycloak)

Pros:

- No laptop performance issues — Railway handles the computing
- Looks more professional (permanent URLs, HTTPS)
- Can demo from any device (phone, tablet, any computer)
- Multiple people can test simultaneously

Cons:

- Internet required (if WiFi drops, presentation stops)
- Railway free tier limits (1GB RAM — Keycloak may struggle)
- Keycloak users lost on restart if PostgreSQL connection has issues

Quick Reference: Local vs Railway Comparison

Item	Localhost	Railway
Django URL	<code>http://localhost:8000</code>	<code>https://government-assets-platform-production.up.railway.app</code>
Keycloak URL	<code>http://localhost:8180</code>	<code>https://keycloak-production-4f96.up.railway.app</code>
Database	Local PostgreSQL (Windows service)	Railway PostgreSQL (managed)
Database name	<code>gov_asset_platform</code>	<code>railway</code> (auto-created)
Database connection	<code>DB_*</code> vars in <code>.env</code>	<code>DATABASE_URL</code> env var
Django server	<code>runserver</code> (auto-reload)	<code>gunicorn</code> (production)
Keycloak command	<code>kc.bat start-dev --http-port=8180</code>	Docker container: <code>kc.sh start-dev</code>
Keycloak database	Local H2 (file-based)	Railway PostgreSQL (same as Django)
Keycloak admin user	<code>superadmin / Admin@123</code>	<code>superadmin / Admin@123</code>
Static files	Django serves them (automatic)	Whitenoise serves them
HTTPS	No (HTTP)	Yes (HTTPS via Railway proxy)
Debug mode	<code>DEBUG=True</code>	<code>DEBUG=False</code>
Internet required?	No (everything on your laptop)	Yes (services are on Railway)
Startup time	~1 minute (start all 3 services)	Already running (24/7)

The `.env` Settings Side by Side

```
# — FOR LOCAL —
DEBUG=True
DB_NAME=gov_asset_platform
DB_USER=postgres
DB_PASSWORD=your_password
DB_HOST=localhost
DB_PORT=5432

KEYCLOAK_SERVER_URL=http://localhost:8180
KEYCLOAK_CLIENT_ID=govasset-django
KEYCLOAK_CLIENT_SECRET=...
KEYCLOAK_ADMIN_USERNAME=superadmin
KEYCLOAK_ADMIN_PASSWORD=Admin@123
ALLOWED_HOSTS=localhost,127.0.0.1,0.0.0.0
PLATFORM_BASE_URL=http://localhost:8000
# No DATABASE_URL

# — FOR RAILWAY —
DEBUG=False
# Not used (uses DATABASE_URL)
# Not used
# Not used
# Not used
# DATABASE_URL=postgresql://...
KEYCLOAK_SERVER_URL=https://keycloak-...
KEYCLOAK_CLIENT_ID=govasset-django
KEYCLOAK_CLIENT_SECRET=...
KEYCLOAK_ADMIN_USERNAME=superadmin
KEYCLOAK_ADMIN_PASSWORD=Admin@123
ALLOWED_HOSTS=.up.railway.app
PLATFORM_BASE_URL=https://goverment-...
DATABASE_URL=postgresql://...
```

Troubleshooting Localhost Issues

"DisallowedHost at /"

Problem: You see `Invalid HTTP_HOST header: 'localhost:8000'`. You may need to add `'localhost'` to `ALLOWED_HOSTS`.

Fix: Add to your `.env`:

```
ALLOWED_HOSTS=localhost,127.0.0.1,0.0.0.0
```

"could not connect to server: Connection refused"

Problem: PostgreSQL is not running.

Fix:

```
net start postgresql-x64-17
```

"Keycloak: Port already in use"

Problem: Port 8180 is occupied by another program.

Fix:

```
netstat -ano | findstr :8180
taskkill /PID <number> /F
```

"No realm 'govasset' found"

Problem: You started Keycloak but never created the `govasset` realm.

Fix:

1. Open `http://localhost:8180/admin`
2. Login: `superadmin / Admin@123`
3. Hover over the realm dropdown (top-left, shows "master")
4. Click "Create realm"
5. Name: `govasset`
6. Create
7. Create the OIDC client: `govasset-django`

"User does not exist" when logging into SSO

Problem: User exists in Django but not in Keycloak (or vice versa).

Fix:

1. Open Keycloak admin: `http://localhost:8180/admin`
2. Switch to `govasset` realm
3. Users → Add user → Create the user
4. Credentials → Set password → `Admin@123` (Temporary: OFF)

"Token not valid" or 401 from verify-token

Problem: Keycloak Client Secret doesn't match between Django and Keycloak.

Fix:

1. Open Keycloak admin → `govasset` realm
2. Clients → `govasset-django` → Credentials tab
3. Copy the Client Secret
4. Paste it into your `.env`: `KEYCLOAK_CLIENT_SECRET=...`
5. Restart Django

"Invalid redirect URI" on local Keycloak

Problem: Keycloak doesn't recognize `http://localhost:8000/*` as a valid redirect.

Fix:

1. Open Keycloak admin → `govasset` realm
2. Clients → `govasset-django` → Settings
3. Under "Valid redirect URIs" add: `http://localhost:8000/*`

4. Under "Valid post logout redirect URIs" add: `http://localhost:8000/*`
5. Save

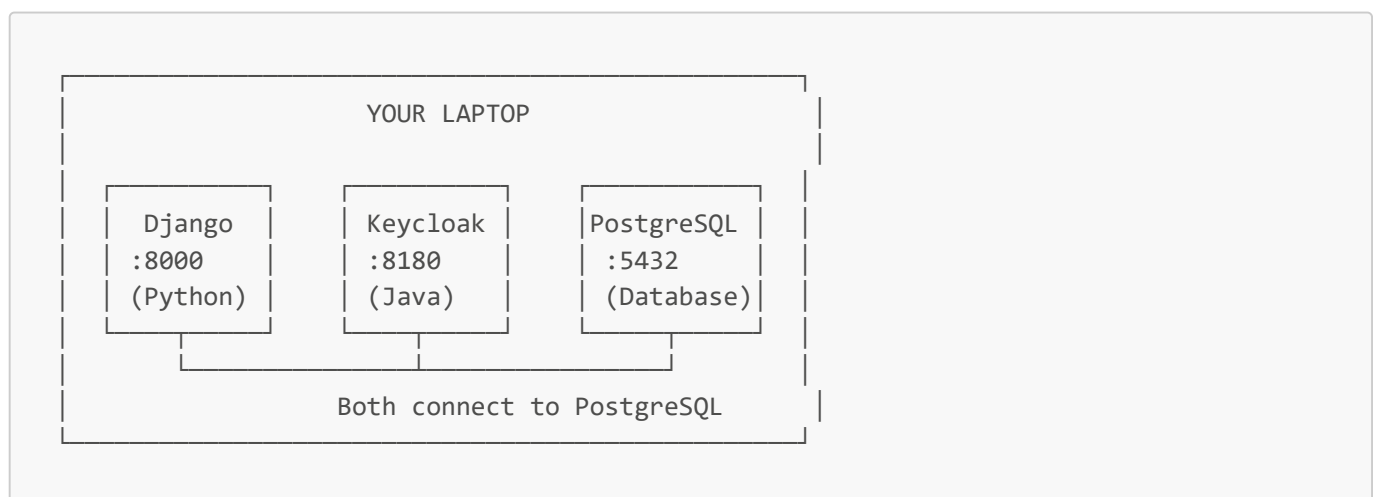
Summary

Question	Answer
What code changes did we make?	<code>settings.py</code> (ALLOWED_HOSTS, DATABASE_URL, SECURE_PROXY_SSL_HEADER, PLATFORM_BASE_URL, whitenoise), <code>views.py</code> (logout URL), <code>setup_demo_data.py</code> (ministries + tenant migrations)
Do I need to undo any code changes for local?	No — the code is designed to work in both environments automatically
What determines local vs Railway?	Your <code>.env</code> file (local) vs Railway Variables tab (online)
Do I need to change <code>.env</code> between local and Railway?	Yes — but you only change it when switching. If you're presenting locally, use the local <code>.env</code> . If presenting via Railway, you don't touch <code>.env</code> at all (Railway uses its own Variables tab).
Can I run both at the same time?	Yes — they use separate databases and separate Keycloaks. Changes on local don't affect Railway.
What if I accidentally delete Railway Keycloak?	You'll need to recreate the realm and users via the admin console. The user passwords remain in Django DB but Keycloak accounts are lost.

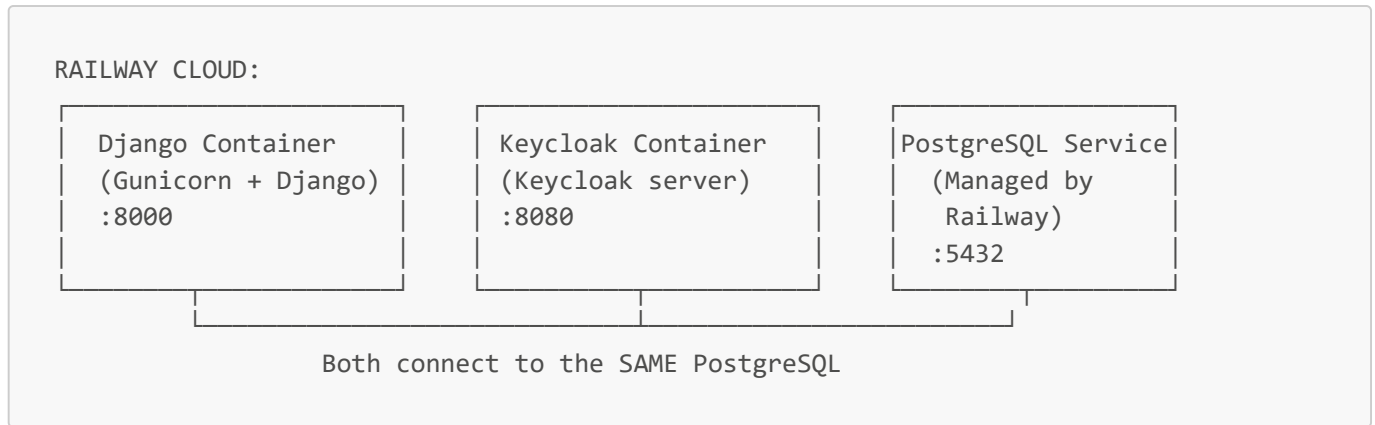
Database Access — Viewing Your Data

What is PostgreSQL and where does it run?

PostgreSQL is a **database program** — a separate piece of software from Django. It stores all your data: users, assets, audit logs, and (on Railway) Keycloak data too.



On **Railway**, it's the same but each service is in its own container:



Key insight: On Railway, Django AND Keycloak share one PostgreSQL database. Django uses the **railway** database. Keycloak also uses the same **railway** database but creates its own tables (like **realm**, **client**, **credential**, **user_entity**) right alongside Django's tables.

Three Ways to Access the Database

Method 1: Django's built-in dbshell (Easiest)

Locally:

```
python manage.py dbshell
```

This opens the **psql** command-line tool automatically connected to your database.

On Railway:

1. Open Railway Dashboard → Django service → Connect → Web Terminal
2. In the terminal:

```
python manage.py dbshell
```

What you see:

```
psql (16.x)
Type "help" for help.

railway=>
```

Now you can type SQL commands:

```
-- List all schemas (each ministry = one schema)
\dn

-- You should see:
-- public          ← Shared tables (users, tenants, settings)
```

```
-- moh_schema    ← Ministry of Health assets
-- mof_schema    ← Ministry of Finance assets

-- List tables in the public schema
\dt public.*

-- List tables in a specific ministry schema
\dt moh_schema.*

-- See what users exist and their roles
SELECT id, username, role, ministry_schema FROM public.authentication_customuser;

-- Count assets per ministry
SELECT COUNT(*) FROM moh_schema.assets_asset;
SELECT COUNT(*) FROM mof_schema.assets_asset;

-- Exit
\q
```

Method 2: Desktop Tool (pgAdmin, DBeaver, DataGrip)

This is what most people imagine when they say "open the database" — a graphical interface where you see tables, click them, and browse data.

For your LOCAL database:

Setting	Value
Host	localhost
Port	5432
Database	gov_asset_platform
User	postgres
Password	(your local PostgreSQL password)

For your RAILWAY database:

Get the connection details:

1. Railway Dashboard → PostgreSQL service → Connect tab
2. You'll see the connection string:

```
postgresql://postgres:abc123@containers-us-west-xxx.railway.app:5432/railway
```

3. Extract these values for pgAdmin/DBeaver:

Setting	Value
Host	containers-us-west-xxx.railway.app

Setting	Value
Port	5432
Database	railway
User	postgres
Password	abc123

WARNING about Railway PostgreSQL:

- Railway rotates the password sometimes (when you restart the service)
- Railway's PostgreSQL is only accessible from Railway's internal network OR via the public proxy
- Use the "Connect" tab to always get the latest credentials
- Some desktop tools may not work because Railway restricts external connections — in that case, use Method 1 (Web Terminal)

Method 3: Direct SQL via Railway Web Terminal

In Railway → PostgreSQL service → Connect → Web Terminal:

```
-- You are directly in psql
\dt
SELECT * FROM public.authentication_customuser;
```

Understanding the Schema Structure

Our database has ONE database with MULTIPLE schemas:

```
Database: gov_asset_platform (local) or railway (Railway)
|
|— Schema: public (shared data – all ministries)
|   |— authentication_customuser      ← All users from all ministries
|   |— tenants_ministry               ← List of ministries
|   |— tenants_domain                 ← Domain-to-ministry mapping
|   |— django_session                 ← Login sessions
|   |— authentication_loginattempt    ← Failed login tracking
|   |— authentication_unlocktoken     ← Password unlock tokens
|   |— authentication_pendingaccess   ← Users awaiting approval
|   |— django_migrations              ← Which migrations ran
|   |— django_admin_log               ← Admin actions
|   |— django_content_type            ← Django internals
|
|— Schema: moh_schema (Ministry of Health – ISOLATED)
|   |— assets_asset                   ← Health ministry assets
|   |— assets_assetcategory           ← Asset categories
|   |— organizations_orgunit          ← Hospitals, clinics
|   |— organizations_masterdata       ← Dropdown options
|   |— organizations_auditlog         ← Tamper-proof audit trail
|
|— Schema: mof_schema (Ministry of Finance – ISOLATED)
```

```

├── assets_asset                ← Finance ministry assets
├── assets_assetcategory
├── organizations_orgunit
├── organizations_masterdata
├── organizations_auditlog
└── Schema: (Keycloak tables – Railway only, in public schema)
    ├── realm                  ← The govasset realm
    ├── client                  ← OIDC clients
    ├── user_entity             ← Keycloak users
    ├── credential              ← Passwords (hashed)
    └── ... (many more Keycloak tables)

```

Why this matters:

- A SQL query `SELECT * FROM assets_asset` would fail with "relation not found" because you must specify the schema
- You must write: `SELECT * FROM moh_schema.assets_asset`
- This is how multi-tenancy works — each ministry's data is physically separated at the database level

Useful SQL Commands for the Presentation

```

-- Show the judges that data is isolated per ministry
SELECT 'Ministry of Health' AS ministry, COUNT(*) AS assets
FROM moh_schema.assets_asset
UNION ALL
SELECT 'Ministry of Finance', COUNT(*)
FROM mof_schema.assets_asset;

-- Show all users and their roles
SELECT username, role, ministry_schema, is_active
FROM public.authentication_customuser
ORDER BY role;

-- Show recent audit log entries (tamper-proof)
SELECT * FROM moh_schema.organizations_auditlog
ORDER BY timestamp DESC
LIMIT 10;

-- Show how schemas look in the database
SELECT schema_name
FROM information_schema.schemata
WHERE schema_name NOT LIKE 'pg_%'
  AND schema_name NOT LIKE 'information_schema';

```

What to Say to the Judges About the Database

If a judge asks "How does your database work?" or "Show me the schema":

"Our database uses PostgreSQL with a multi-tenant architecture. Each ministry gets its own schema — think of a schema as a

separate folder. When a user logs in, Django sets the `search_path` to their ministry's schema. So even if there is a bug in our code, the database itself prevents one ministry from seeing another's data.

Here, I'll show you: (open `dbshell`)

```
\dn - These are our schemas. public for shared tables,  
      moh_schema for Ministry of Health, mof_schema for Finance.
```

If I query the `user` table in `public` schema:

```
SELECT username, role, ministry_schema FROM public.authentication_customuser;
```

And if I count assets per schema:

```
SELECT 'MOH' AS ministry, COUNT(*) FROM moh_schema.assets_asset  
UNION ALL  
SELECT 'MOF', COUNT(*) FROM mof_schema.assets_asset;
```

Each ministry's data is completely isolated at the database level."

Users, Roles, and PendingAccess — Two Worlds That Must Match

The Big Confusion: Keycloak Users vs Django Users

Beginners always get confused because there seem to be TWO places where users exist. Here is the truth:

KEYCLOAK USERS

Stored in Keycloak's database
(or the shared PostgreSQL on Railway)

Purpose: IDENTITY

"Who are you?"

Username + password

Contains:

- Username
- Password (hashed)
- Email
- Name

DJANGO USERS

Stored in Django's database
(`public.authentication_customuser`)

Purpose: AUTHORIZATION

"What can you do?"

Role + Ministry

Contains:

- Username
- Role (`SUPER_ADMIN`, etc.)
- Ministry (`moh_schema`, etc.)
- Keycloak ID (link back)
- `is_active`, `is_locked`
- Django-specific permissions

A user must exist in BOTH places to log in through SSO.

User logs in:

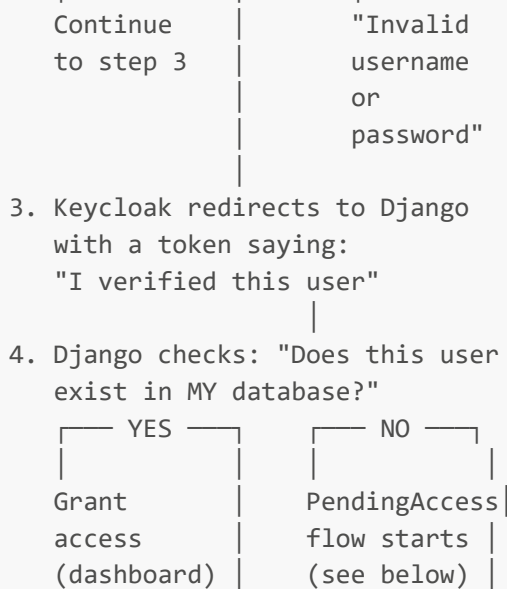
1. User types username + password on Keycloak page

|

2. Keycloak checks: "Does this user exist in MY database?"

┌ YES ─┐

┌ NO ─┐



The Two Types of Roles

There are TWO separate role systems:

Role System	Where it lives	What it controls	Given by
Keycloak roles	Keycloak admin console	Access to Keycloak itself (admin console)	Admin in master realm
Django roles	Django database (<code>authentication_customuser.role</code>)	Access to platform features	Super Admin in Django admin

Example: A Health Ministry employee:

```

Keycloak role:  none (or "user")    → Can log in, cannot access admin console
Django role:    MINISTRY_ADMIN      → Can manage Health assets in the platform
  
```

Example: A Platform Administrator:

```

Keycloak role:  admin (in master realm) → Can access Keycloak admin console
Django role:    SUPER_ADMIN             → Can manage all ministries in the platform
  
```

The PendingAccess Flow (User in Keycloak but Not Django)

This is our solution to a real-world problem:

Scenario: A government IT officer creates a user in Keycloak (the central identity system) but forgets to tell us. The user tries to log in to our platform.

```
User exists in Keycloak: YES
User exists in Django: NO
```

What happens:

1. User logs in on Keycloak page → SUCCESS (Keycloak knows them)
2. Keycloak redirects to Django with valid token
3. Django checks database: no user found
4. Django does NOT crash. Instead:
 - a. Creates a PendingAccess record
 - b. Shows the user a message: "Your account is pending approval. Contact your system administrator."
 - c. A notification appears in the Django admin panel for Super Admins to review
5. Super Admin sees the pending request, approves it, assigns a role and ministry
6. Django creates the user in its database
7. User can now log in normally

Where this is configured:

- `authentication/oidc_backend.py` — The code that handles this flow
- `authentication/models.py` — The `PendingAccess` model
- Django admin panel — Where Super Admins review and approve requests

What You Need to Do in Keycloak for Each User

When you create a user in your local Keycloak admin, you need to assign them to the correct realm role:

Step 1 — Create the user in Keycloak:

1. Keycloak admin → govasset realm → Users → Add user
2. Username: (match the Django username)
3. Email: (optional)
4. Save

Step 2 — Set password:

1. Credentials tab
2. Set password: Admin@123
3. Temporary: OFF
4. Save

Step 3 — Assign realm role (important for SSO):

1. Role mapping tab
2. Assign role → Select "user" (or "offline_access" if needed)

3. Add

What role should you assign in Keycloak?

| For regular users | Assign **user** role | | For offline access | Assign **offline_access** + **user** |

Note: The Keycloak role is just for Keycloak's internal permission system. The ACTUAL permission (SUPER_ADMIN, MINISTRY_ADMIN, etc.) comes from Django's database, not Keycloak.

Step 4 — Set custom attributes (REQUIRED for Django OIDC to work):

1. Attributes tab
2. Add the following attributes (click "Add" for each):

Attribute key	Attribute value
role	e.g. SUPER_ADMIN, MINISTRY_ADMIN, AGENCY_MANAGER, FACILITY_CLERK, AUDITOR
ministry_schema	e.g. moh_schema, mof_schema (leave blank for SUPER_ADMIN)

3. Save

Why this matters: Django's **oidc_backend.py** reads these attributes from the Keycloak token on every login. If they are missing:

- The user's Django role won't auto-sync (they'll get the default empty role)
- **ministry_schema** won't be populated on their **PendingAccess** record, so the approving Super Admin will have to guess which ministry they belong to
- You'll see warnings in the Django logs like **"No role attribute found for user X"**

Tip: If you're using **setup_demo_data** to create users, it sets these attributes automatically. You only need to worry about this when creating users manually in the Keycloak admin console.

How to Verify a User Has Proper Access

If someone says "I can log in but I can't see anything," follow these steps:

1. Can they log into Keycloak?
 - Test: Open Keycloak login page, try username + password
 - If fails: User doesn't exist in Keycloak → Create them
2. Can they log into Django (SSO redirects back)?
 - If redirected back with error: User doesn't exist in Django
 - Check PendingAccess in Django admin
 - Approve and assign role
3. Do they see the dashboard but no data?
 - Check their Django role and ministry
 - Query:

```
SELECT username, role, ministry_schema
FROM public.authentication_customuser
WHERE username = 'their_username';
```

- If role is empty or wrong → Fix in Django admin
- If ministry is wrong → Fix in Django admin

Changing Domain or Host Provider

What Happens If You Change Your Domain

If you change from `railway.app` to a custom domain (like `govasset.go.tz`), or switch from Railway to another provider entirely — many things break. Here is everything that needs updating.

The Complete List of Every File/Config That References the Domain

Category 1: Django Python Code (settings.py)

File	Line/Setting	Current value	What to change
<code>config/settings.py</code>	<code>PLATFORM_BASE_URL</code>	<code>config('PLATFORM_BASE_URL', default='http://localhost:8000')</code>	Change the default OR update the env var on the new server
<code>config/settings.py</code>	<code>ALLOWED_HOSTS</code>	<code>config('ALLOWED_HOSTS').split(',')</code>	Update the env var on the new server
<code>config/settings.py</code>	<code>CSRF_TRUSTED_ORIGINS</code> (optional)	May contain old domain	Update to new domain
<code>config/settings.py</code>	<code>CORS_ALLOWED_ORIGINS</code> (if set)	May contain old domain	Update to new domain
<code>authentication/views.py</code>	<code>logout_view</code>	Uses <code>settings.PLATFORM_BASE_URL</code>	✓ No change needed — uses env var

Category 2: Environment Variables (Need Changing)

If you move to a new provider, you MUST set these env vars on the new server:

```
# Django env vars to update:
ALLOWED_HOSTS=.newdomain.com
PLATFORM_BASE_URL=https://your-app.newdomain.com
KEYCLOAK_SERVER_URL=https://keycloak.newdomain.com
CSRF_TRUSTED_ORIGINS=https://your-app.newdomain.com

# Keycloak env vars to update:
KC_HOSTNAME=https://keycloak.newdomain.com
KC_PROXY=edge          ← Keep this for any cloud provider behind a proxy
```

Category 3: OIDC Clients in Keycloak (Important!)

Every client you created in Keycloak has URL settings that must match your actual URLs:

Client	What to update in Keycloak admin
govasset-django	Settings → Valid Redirect URIs: Add new domain
govasset-django	Settings → Valid Post Logout Redirect URIs: Add new domain
govasset-django	Settings → Web Origins: Add new domain
govasset-django	Settings → Root URL: Update to new domain
account-console	Settings → Web Origins: Add new domain
Any external system client (e.g., group2-app)	All redirect URIs must be updated

If you don't update these: Users will see "Invalid redirect URI" errors when trying to log in.

Category 4: External Systems

Every external system (Group 2, Group 5, etc.) has our URLs hardcoded or configured in their systems:

What they have:	What changes:
KEYCLOAK_URL=https://keycloak-...	→ https://keycloak.newdomain.com
API_URL=https://government-assets-...	→ https://api.newdomain.com
CALLBACK_URL=https://their-system...	→ No change (their URL stays same)

You must notify every external group and give them the new URLs.

Category 5: External Documentation

File	What references the old domain
EXTERNAL_SSO_INTEGRATION.md	Keycloak URL, API URL, sample code
direct_api_integration.md	API URL in sample code
presentation_qa.md	Everywhere Railway URLs appear (many places)
LOCAL_VS_ONLINE_GUIDE.md	Railway URLs in comparison tables

File	What references the old domain
KEYCLOAK_ADMIN_GUIDE.md	Admin console URL

Step-by-Step: Moving from Railway to Another Provider

PHASE 1: SET UP THE NEW INFRASTRUCTURE

Step 1: Choose a new provider

Options: DigitalOcean, AWS, Azure, VPS (Linode, Hetzner),
or on-premise server

Step 2: Provision the new server

Minimum specs: 2GB RAM, 2 CPU cores, 20GB SSD
OS: Ubuntu 22.04 or 24.04 LTS

Step 3: Install required software

- PostgreSQL 16+
- Python 3.11+
- Java 17+ (for Keycloak)
- Nginx (as reverse proxy)
- Certbot (for HTTPS certificates)
- Git

Step 4: Set up PostgreSQL

- Create database and user
- Restore from backup:


```
pg_dump -U postgres -h old-host railway > backup.sql
psql -U postgres -h new-host -d gov_asset_platform < backup.sql
```

Step 5: Deploy Django

- Clone GitHub repo on new server
- Set up virtual environment
- Install requirements
- Create .env file with NEW values
- Run: `gunicorn config.wsgi --bind 0.0.0.0:8000`

Step 6: Deploy Keycloak

- Download Keycloak or use Docker
- Set env vars with NEW domain
- Restore realm export (if you exported it)
- Start Keycloak

Step 7: Set up Nginx reverse proxy

```
server {
    listen 443 ssl;
    server_name your-app.newdomain.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
}
```

(Similar config for Keycloak on a different subdomain)

Step 8: Get HTTPS certificates

```
certbot --nginx -d your-app.newdomain.com
```

Step 9: Update DNS

Point your domain's A records to the new server's IP address

PHASE 2: UPDATE ALL CONFIGURATIONS

Step 10: Update Keycloak client redirect URIs

- govasset-django: add new domain to Valid Redirect URIs
- account-console: add new domain to Web Origins

Step 11: Update Django env vars on new server

```
ALLOWED_HOSTS=.newdomain.com
PLATFORM_BASE_URL=https://your-app.newdomain.com
KEYCLOAK_SERVER_URL=https://keycloak.newdomain.com
```

Step 12: Update all external groups

Send them the new Keycloak URL and API URL

Step 13: Update all documentation files

Search for "railway.app" and replace with new domain

PHASE 3: SWITCH OVER

Step 14: Test everything on the new server

- Can you log in?
- Can you see data?
- Can you access Keycloak admin?

Step 15: Switch DNS

- Update DNS records to point to the new server
- Wait for DNS propagation (can take 24-48 hours)

Step 16: Keep old Railway running for 1 week

- Some users may still have old DNS cached
- Monitor both systems

Step 17: Shut down Railway services

- Only after confirming all traffic has switched

The Domain Change Summary Table

What breaks if domain changes	Fix
Django ALLOWED_HOSTS	Update env var on new server

What breaks if domain changes	Fix
Django logout redirect	Update PLATFORM_BASE_URL env var
Keycloak redirects	Update KC_HOSTNAME env var + restart Keycloak
OIDC login (Invalid redirect URI)	Update govasset-django client in Keycloak admin
Manage Account page	Update account-console client Web Origins
External systems can't log in	Give them new URLs, they update their configs
Group 2 integration broken	They update their code with new URLs
HTTPS certificate	Set up new cert for new domain
Database	Migrate/backup/restore to new PostgreSQL
All documentation	Search and replace old URLs

What Does NOT Break

These things survive a domain change without any modification:

- Your Django code (Python files) — as long as you use env vars
- Your database schema and data (if you migrate properly)
- User passwords in Keycloak (if you export/import the realm)
- Role assignments in Django
- Asset data in ministry schemas
- Audit logs

How Railway Actually Works (So You Can Explain It)

When a judge asks "How does Railway host your application?":

THE RAILWAY MECHANISM:

1. You connect your GitHub repository to Railway
2. Railway detects it's a Python project (requirements.txt)
3. Railway builds a Docker container:
 - Takes the official Python image
 - Copies your code into it
 - Runs: `pip install -r requirements.txt`
 - Starts: `gunicorn config.wsgi`
4. Railway runs this container on their servers
5. Railway provides:
 - A public URL: `*.up.railway.app`
 - HTTPS automatically (Let's Encrypt)
 - PostgreSQL as a managed service
 - Environment variables from the Variables tab
 - Automatic restarts if the container crashes

For Keycloak:

1. Railway pulls the official Keycloak Docker image
2. Sets env vars from the Variables tab
3. Runs the container with: `/opt/keycloak/bin/kc.sh start-dev`

4. Keycloak connects to the same PostgreSQL as Django

The magic: Every time you push code to GitHub, Railway automatically builds and redeploys Django.

What Judges May Ask About Hosting

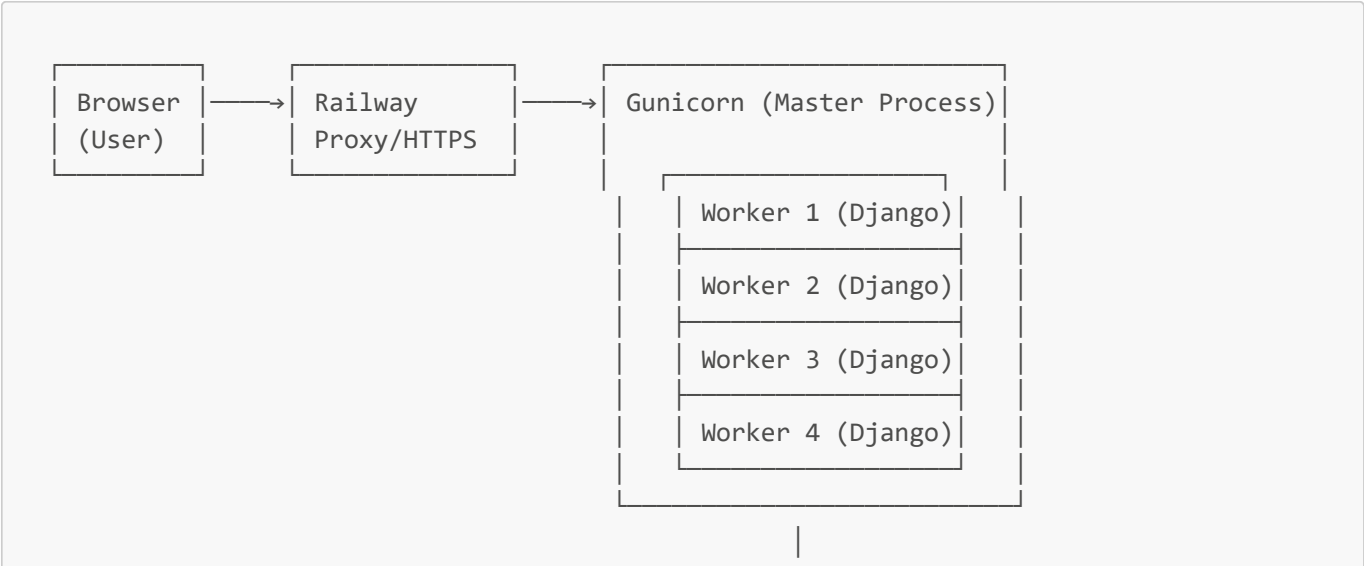
Judge's question	How to answer
"Why did you choose Railway?"	"Railway was chosen for the prototype because it offers a free tier with PostgreSQL, automatic HTTPS, and simple GitHub deployment. It requires no server management — ideal for a university project."
"What are Railway's limitations?"	"The free tier has 1GB RAM, which limits Keycloak's performance. PostgreSQL is shared. There is no guaranteed uptime SLA. For production, we would migrate to a dedicated server or cloud provider."
"How would you handle 10,000 users?"	"We would move to a provider like DigitalOcean or AWS. Add a load balancer with multiple Django instances. Use connection pooling for PostgreSQL. Add Redis for caching. Consider managed Keycloak (Keycloak as a Service)."
"What is your backup strategy?"	"Currently, we rely on Railway's PostgreSQL backups. For production, we would add daily automated backups using pg_dump, stored in encrypted cloud storage. Keycloak realm exports would be done weekly."
"How is security handled?"	"HTTPS is automatic on Railway. Django uses settings like SECURE_PROXY_SSL_HEADER, CSRF protection, and CORS. Passwords are hashed. Brute force protection is enabled. Audit logs are tamper-proof at the database level."

Advanced Concepts for Deep Understanding

How Django Handles Requests in Production (Gunicorn)

When you run `python manage.py runserver`, Django handles ONE request at a time. This is fine for development but terrible for production.

In production (Railway), Django runs behind **Gunicorn**:





PostgreSQL

Gunicorn workers: Each worker is a separate Django process that can handle one request at a time. With 4 workers, Django can handle 4 requests simultaneously. Railway likely auto-configures this.

Why this matters:

- **runserver** handles 1 request at a time (for development only)
- Gunicorn handles multiple requests in parallel (for production)
- If you have 4 workers and 5 users make requests at the same time, 4 are served immediately and 1 waits

Static Files in Production

In development, Django automatically serves your CSS, JavaScript, and images. In production, Django does NOT serve static files by default — this is a security feature.

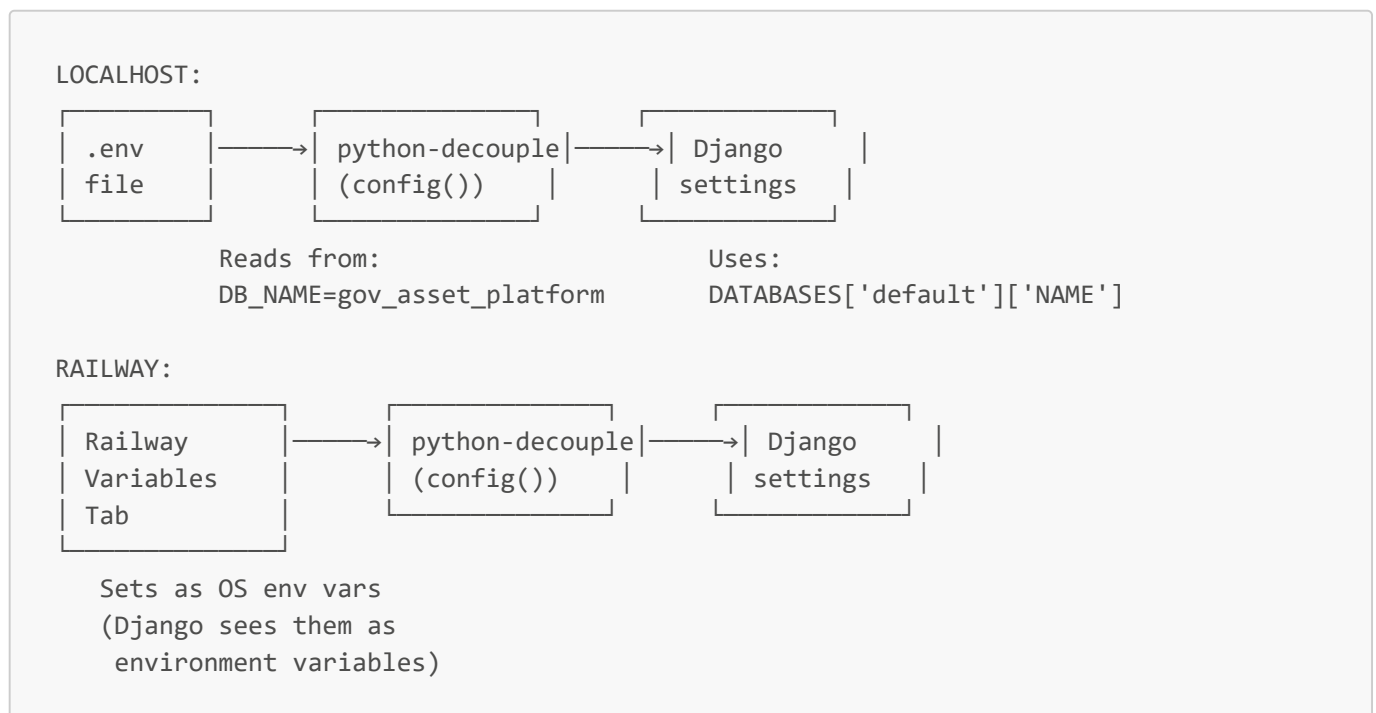
How we solved it: We added **whitenoise** to **requirements.txt** and added **WhiteNoiseMiddleware** to **MIDDLEWARE**.

Whitenoise intercepts requests for static files and serves them efficiently. It compresses files (gzip) and sets far-future cache headers (so browsers don't re-download your CSS on every page load).

Without whitenoise: Your production site would have NO CSS — it would look like a plain HTML page with no styling.

Environment Variables Flow

This diagram shows how env vars travel from their source to your code:



Key difference:

- Locally: `.env` file is read by `python-decouple`
- Railway: Variables Tab sets actual OS environment variables
- The code (`config('VAR_NAME')`) works the same way in both cases

How GitHub Deployments Work

When you push code to GitHub and see changes on your website:

```
1. You edit code on your laptop
   |
2. You commit and push to GitHub:
   git add .
   git commit -m "Fixed login bug"
   git push origin main
   |
3. GitHub now has your new code
   |
4. Railway detects the push
   (it watches your GitHub repo)
   |
5. Railway builds a new container:
   - Installs requirements.txt
   - Runs collectstatic
   - Prepares the application
   |
6. Railway deploys the new container
   - Takes the old container down
   - Starts the new container
   - This takes ~30-60 seconds
   |
7. Your website is now updated
   with your changes

DOWNTIME: During step 6, there may be
a few seconds where your site is
unavailable. Railway handles this
gracefully – active users don't lose data.
```

Important: This only applies to Django. Keycloak is deployed as a Docker image — it only updates when you change the image tag or env vars, not when you push code to GitHub.

HTTPS and SSL Explained Simply

Term	Meaning
HTTP	Unencrypted web traffic — anyone on your WiFi can see the data
HTTPS	Encrypted web traffic — only the browser and server can read it
SSL/TLS	The technology that encrypts the connection
Certificate	A digital ID proving "this server is really who it says it is"
Certificate Authority	A company (like Let's Encrypt) that issues certificates

On Railway: HTTPS is automatic. Railway gets a certificate for your `*.up.railway.app` URL from Let's Encrypt. You don't have to do anything.

On your own server: You'd use Certbot to get a free certificate from Let's Encrypt:

```
sudo certbot --nginx -d your-domain.com
```

Why HTTPS matters for SSO: If you don't have HTTPS, browsers block the redirect from Keycloak back to your app. The user would see a warning page saying "This connection is not secure."

Keycloak's Internal Architecture

Keycloak is a Java application that runs on **Quarkus** (a lightweight Java framework). When you start it:

```
kc.bat start-dev
  |
  ↓
Quarkus starts a Java Virtual Machine (JVM)
  |
  ↓
Keycloak loads its configuration (env vars)
  |
  ↓
Keycloak connects to the database
  |
  ↓
Keycloak starts an HTTP server on port 8180
  |
  ↓
Keycloak is now ready to handle login requests
```

Why Java matters:

- Java needs more memory than Python (hence the `JAVA_OPTS_APPEND=-Xmx256m`)
- Keycloak takes 20-40 seconds to start (Java startup time)
- Keycloak consumes ~200-300MB RAM at idle, more under load
- This is why Railway's 1GB free tier is barely enough

Database Connection Pooling

Every time Django handles a request, it opens a new database connection. Opening a connection takes time and resources.

In development (`runserver`), each request opens a new connection and closes it when done.

In production (Gunicorn), each worker opens a connection on startup and **reuses it** for multiple requests. This is called **connection pooling** — it's much faster.

PostgreSQL has a maximum number of connections (default: 100). With 4 Gunicorn workers, you use 4 connections. With 10 workers, you use 10 connections. When all connections are in use, new requests wait until

one is freed.

On Railway: This is managed automatically. You don't need to worry about it.

The Complete Data Flow (End to End)

Here is what happens when a user clicks "Login" on your platform:

USER'S BROWSER:

1. User clicks "Login"
2. Browser URL changes to:
`http://localhost:8000/login/`

DJANGO:

3. Django receives the request
4. mozilla-django-oidc plugin intercepts
5. Django generates a login URL:
`http://localhost:8180/realms/govasset/...`
6. Django responds with HTTP 302 (redirect)

USER'S BROWSER (FOLLOWS REDIRECT):

7. Browser navigates to Keycloak's login page
8. User sees: Keycloak login form
9. User types username + password

KEYCLOAK:

10. Keycloak receives credentials
11. Keycloak looks up user in its database
12. Keycloak verifies password hash
13. Valid? → Creates a session
14. Generates an authorization code
15. Redirects user back to Django with the code

DJANGO (CALLBACK HANDLER):

16. Django receives code at `/oidc/callback/`
17. Django POSTs to Keycloak:
"Here's the code, give me tokens"
18. Keycloak verifies the code, sends back tokens
19. Django verifies token signature (cryptographic)
20. Django looks up user in its database
21. Found? → Creates Django session
22. Not found? → Creates PendingAccess record
23. If found: redirects to dashboard

USER'S BROWSER:

24. Sees dashboard: "Welcome, Amina Hassan"
25. All data is filtered by ministry and role

Deep Dive: Everything That Confuses People About Deployment

This section answers every question you had about how things work after deployment. Read it in order — each answer builds on the previous one.

Q1: "Can I access the code offline in my VS Code?"

Yes. Your project is physically stored on your laptop at `D:\government_asset_platform`. You can open VS Code, edit any file, add new features, change the UI — everything works offline exactly as before.

```
Your Laptop (VS Code):
D:\government_asset_platform\
├─ config/settings.py      ← You edit this
├─ templates/              ← You edit HTML here
├─ static/css/style.css    ← You edit CSS here
└─ ... all your code

↓ You run locally:
python manage.py runserver
→ Opens at http://localhost:8000
```

Two separate copies exist:

Where	What	How to update
Your laptop	The source code — where you write and edit files	Open VS Code and edit
Railway server	A running copy of your code that serves the website	Push to GitHub → Railway redeploys

Think of it like this: Your laptop is the "workshop" where you build and fix things. Railway is the "store" where customers see the finished product. You build in the workshop, then ship to the store via `git push`.

Q2: "If I change the UI locally (like change a color or fix a button), how does it appear on the hosted website?"

You need to **push the changes to GitHub** so Railway can redeploy.

The workflow:

1. You edit `style.css` in VS Code
 - Change `--accent: #2563eb` to `--accent: #ff0000`
 - Now it's red on your local machine (`localhost:8000`)
2. You commit and push to GitHub:

```
git add static/css/style.css
git commit -m "Change accent color to red"
git push origin main
```
3. Railway detects the push (it watches your repo)
4. Railway builds a new container:
 - Copies your updated `style.css`

- Runs `collectstatic` (prepares CSS for production)
 - Starts the new container
5. Railway swaps the old container for the new one
→ Now <https://your-app.up.railway.app> shows the red accent
6. You refresh your phone → the color changed

The key rule: Changes on your laptop stay on your laptop until you `git push`. Changes pushed to GitHub get deployed to Railway automatically.

If you want to test a UI change quickly without pushing:

- Run `python manage.py runserver` locally
- Open <http://localhost:8000> on your laptop browser
- Your changes appear immediately (Django auto-reloads)
- Only push to GitHub when you're happy with the result

Q3: "How does Keycloak run in Docker if I don't have Docker Desktop installed?"

You didn't install Docker. Railway did.

The distinction:

Environment	How Keycloak runs
Your laptop (localhost)	You run Keycloak directly: <code>kc.bat start-dev</code> — this starts the Java application directly on Windows. No Docker involved.
Railway (production)	Railway runs Keycloak inside a Docker container. Railway has Docker installed on their servers. You don't need Docker on your laptop.

How Railway runs Keycloak without you doing anything:

Railway's Server (you never see this):

Railway's Infrastructure

Docker Container #1: Django

- Python + Gunicorn
- Your code from GitHub

Docker Container #2: Keycloak

- Java + Keycloak JAR
- Official Keycloak image
- From: [quay.io/keycloak/...](https://quay.io/keycloak/)

```
PostgreSQL (Managed Service)
- Not in Docker, managed by
  Railway directly
```

How you set up Keycloak on Railway:

1. In Railway dashboard, you clicked "New" → "Create New Service"
2. You chose "Docker Image" as the source
3. You entered: `quay.io/keycloak/keycloak:26.1.5`
4. Railway downloaded that image and runs it on THEIR servers
5. You never installed Docker on your laptop

What is a Docker image? A pre-packaged bundle containing everything an application needs to run. The Keycloak image contains Java, the Keycloak server files, and configuration. Railway just downloads and runs it.

Q4: "Why is Keycloak in Docker but Django isn't?"

Both are in Docker on Railway. But they're set up differently:

Service	How it runs on Railway	Why this way
Django	Railway builds a Docker container FROM YOUR CODE	Railway reads your code from GitHub, installs requirements, and creates a container. You don't provide a Dockerfile — Railway figures it out.
Keycloak	Railway runs a PRE-BUILT Docker image	Keycloak is a complex Java application. You don't want to build it from source. You use the official image that the Keycloak team already built.
PostgreSQL	Railway's managed database service	Railway runs PostgreSQL for you. You don't manage it. You just get a connection URL.

On your laptop (localhost):

- Django runs via `python manage.py runserver` (no Docker)
- Keycloak runs via `kc.bat start-dev` (no Docker)
- PostgreSQL runs as a Windows service (no Docker)

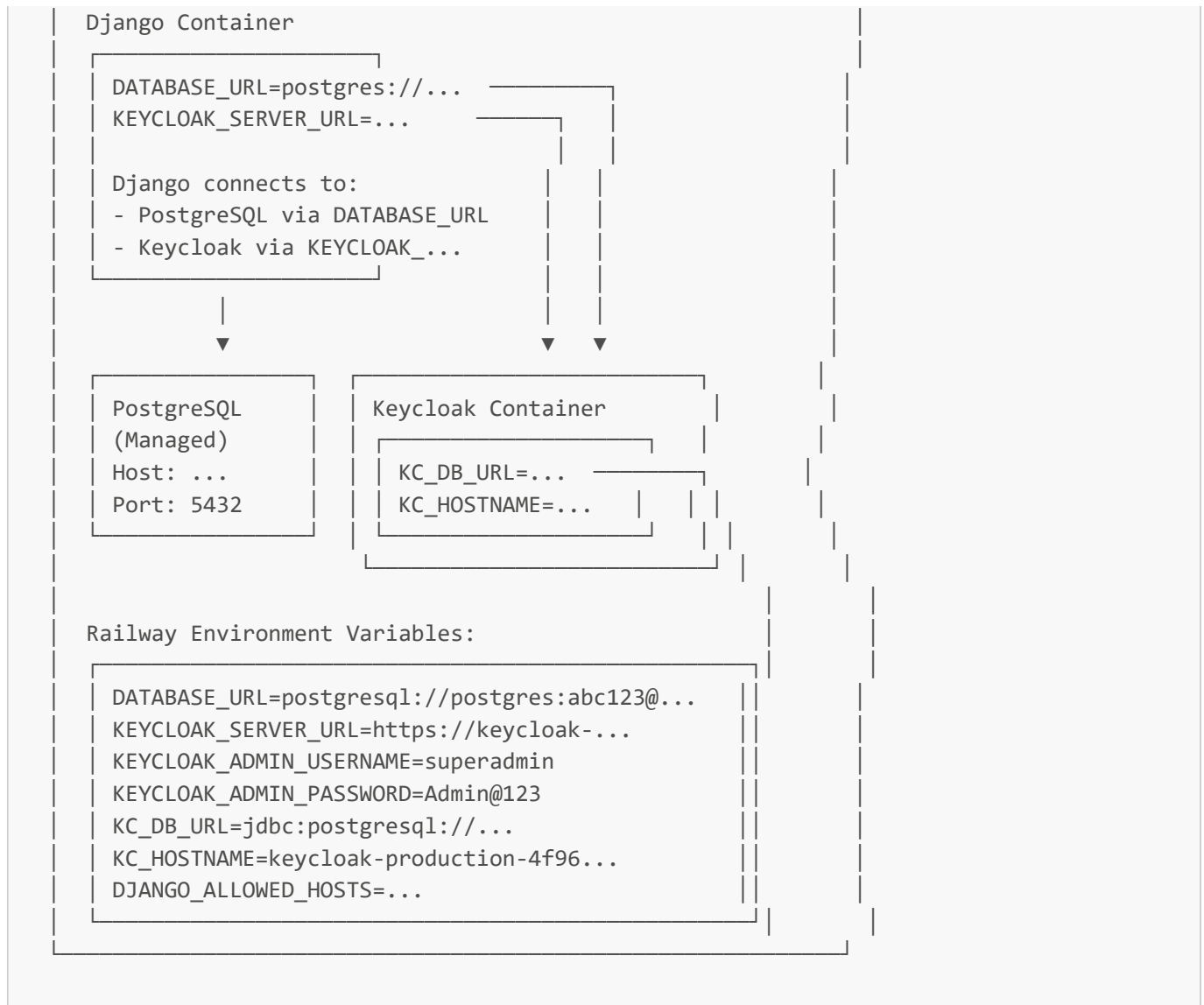
So Docker is ONLY on Railway's servers, never on your laptop.

Q5: "How does Django, PostgreSQL, and Keycloak find each other on Railway?"

They communicate through internal networking and environment variables.

The connection diagram:

Railway Internal Network:



The secret is the Railway Variables Tab. Every service you add to Railway gets a hostname like `django-service-name.up.railway.app` or `keycloak-service-name.up.railway.app`. You put these URLs into the Variables Tab so each service knows where the others are.

Example:

- Railway gives Keycloak the URL: `https://keycloak-production-4f96.up.railway.app`
- You set a variable on Django: `KEYCLOAK_SERVER_URL=https://keycloak-production-4f96.up.railway.app`
- Django reads this variable and knows where to redirect users for login
- Railway gives PostgreSQL a connection string: `postgresql://postgres:abc123@...`
- Both Django and Keycloak use this same connection string to talk to the same database

How Keycloak knows about Django:

- You set `KC_HOSTNAME_URL` on Keycloak's service to tell Keycloak what its own public URL is
- In the Keycloak admin panel, you configured the client `govasset-django` with Django's callback URL: `https://django-service.up.railway.app/oidc/callback/`
- When a user logs in, Keycloak knows to redirect back to Django because of this client configuration

Q6: "How can I access my PostgreSQL database? What is pgAdmin?"

Three ways to access the database:

Method	What it is	How to use
pgAdmin	A desktop app with a graphical interface (point-and-click)	Install pgAdmin, enter the database credentials, see tables, run queries visually
psql	A command-line tool (text only)	Open terminal, type <code>psql -h host -U user -d database</code> , type SQL queries
Railway Dashboard	Web-based database viewer built into Railway	Go to Railway → PostgreSQL service → "Connect" tab → see data, run queries in browser

Is pgAdmin still needed?

Situation	Do you need pgAdmin?
You want to see tables, run SQL queries visually	Yes — pgAdmin is the easiest way
You want to see how many schemas exist	Yes — pgAdmin shows schemas in a tree view
You just want to check if data exists	No — Railway Dashboard is enough
You want to backup or export data	Yes — pgAdmin has backup wizard

How to connect pgAdmin to Railway PostgreSQL:

1. Go to Railway Dashboard → Your PostgreSQL service
2. Click "Connect" tab → Copy the connection string:

```
postgresql://postgres:abc123@hostname.railway.app:5432/railway
```

3. Open pgAdmin → Right-click "Servers" → "Register" → "Server"
4. Fill in:
 - **Name:** Railway PostgreSQL (anything)
 - **Host:** `hostname.railway.app` (the host from the URL)
 - **Port:** `5432`
 - **Username:** `postgres`
 - **Password:** `abc123` (the password from the URL)
5. Click "Save" → you can now browse tables, run queries, see schemas

To see how many schemas exist (for django-tenants):

```
SELECT schema_name FROM information_schema.schemata;
```

This will show:

- `public` — shared tables (users, ministries, etc.)
- `moh_schema` — Ministry of Health data
- `mof_schema` — Ministry of Finance data

- Any other ministry schemas you created

Why can't you "click and open" the PostgreSQL in deployment? Because PostgreSQL is a **database server**, not a website. It doesn't have a web page you open in a browser. It only understands database connections on port 5432. That's why you need pgAdmin (a database client) to connect to it and browse its contents.

Q7: "What is CI/CD? Do we have it?"

CI/CD = Continuous Integration / Continuous Deployment.

Term	What it means	Real-world analogy
CI (Continuous Integration)	Every time you push code to GitHub, the code is automatically tested to make sure nothing is broken	Like an automatic quality check on a factory assembly line
CD (Continuous Deployment)	After tests pass, the code is automatically sent to the production server	Like the factory automatically shipping finished products to the store

Do we have CI/CD?

Type	Do we have it?	How it works
CI	Partially	We have unit tests (83 tests) but they are NOT automatically run on GitHub. You have to run them manually with <code>python -m pytest</code> .
CD	Yes	Every time you <code>git push</code> , Railway detects the change and automatically redeploys Django. This is Continuous Deployment.

What a full CI/CD pipeline looks like (what we don't have yet):

1. You push code to GitHub
|
2. GitHub Actions (CI) triggers automatically:
 - Checks out your code
 - Installs dependencies
 - Runs all 83 tests
 - If tests fail → sends you an email
 - If tests pass → continues
|
3. Railway (CD) triggers:
 - Builds new container
 - Runs migrations
 - Deploys to production
 - If deploy fails → rolls back to previous version
|
4. You get a notification: "Deploy successful"

Why we don't have full CI: Setting up GitHub Actions requires creating a `.github/workflows/test.yml` file that tells GitHub how to run tests. It's not complex but requires configuration. For this project, manual testing is sufficient.

If a judge asks "Do you use CI/CD?":

"We have **continuous deployment** — Railway automatically redeploys whenever we push to GitHub. We don't have **continuous integration** (automated tests on push) set up yet, but our 83 unit tests can be run manually before each deployment to verify nothing is broken. Setting up GitHub Actions to run tests automatically would be the next step."

Q8: "How do I see how many schemas we have?"

Method 1: Using pgAdmin (graphical)

1. Open pgAdmin
2. Connect to Railway PostgreSQL (see Q6 above for connection steps)
3. Expand your server → "Databases" → "railway" → "Schemas"
4. You'll see: `public`, `moh_schema`, `mof_schema`, etc.

Method 2: Using Django shell (command line with code access)

```
python manage.py shell
```

```
from django_tenants.utils import get_tenant_model
for tenant in get_tenant_model().objects.all():
    print(f"{tenant.name} → Schema: {tenant.schema_name}")
```

Output:

```
Ministry of Health → Schema: moh_schema
Ministry of Finance → Schema: mof_schema
```

Method 3: Using Django admin (web browser)

1. Log in as superadmin
2. Go to `/admin/tenants/ministry/`
3. You'll see a list of all ministries with their schema names

Method 4: Via Railway's PostgreSQL CLI (no pgAdmin needed)

```
# Install PostgreSQL CLI tools first
# Then connect directly to Railway's database:
psql "postgres://postgres:abc123@hostname.railway.app:5432/railway"
```

```
# Then run:
SELECT schema_name FROM information_schema.schemata;
```

Q9: "Why can't I just open my database in a browser like a website?"

Because PostgreSQL is a **database management system**, not a web server.

System	What it does	How you access it
Django	Serves web pages	Browser: https://your-app.up.railway.app
Keycloak	Handles authentication	Browser: https://keycloak-xxx.up.railway.app
PostgreSQL	Stores data	Only via database tools: pgAdmin, psql, or code (Django ORM)

What IS accessible in a browser on Railway:

- <https://government-assets-platform-production.up.railway.app> — Django (your app)
- <https://keycloak-production-4f96.up.railway.app> — Keycloak admin (if you enable it)

What is NOT accessible in a browser:

- PostgreSQL — it only speaks the PostgreSQL protocol on port 5432, not HTTP on port 80

Think of it like a phone system:

- Django is like a receptionist who answers calls and talks to customers (your browser requests)
- PostgreSQL is like a filing cabinet in the back room — only the receptionist (Django) can open it
- pgAdmin is like giving you a key to the filing cabinet so you can look at the files directly

Q10: "How does pushing code to GitHub update the live website?"

The full chain explained simply:

```
Step 1: You in VS Code
├─ Edit templates/dashboard.html (change a heading)
├─ Edit static/css/style.css (change a color)
└─ Save the files
    |
Step 2: You open terminal
├─ git add .                (stage all changes)
├─ git commit -m "Update UI" (save a snapshot)
└─ git push origin main     (upload to GitHub)
    |
Step 3: GitHub receives the code
├─ Your repository now has the new files
    |
Step 4: Railway notices the push
├─ Railway is connected to your GitHub repo
├─ It sees: "New commit on main branch"
└─ It starts the deployment process
    |
```

```

Step 5: Railway builds a new container
├─ Downloads your code from GitHub
├─ Installs dependencies: pip install -r requirements.txt
├─ Prepares static files: python manage.py collectstatic
└─ Creates a fresh Docker container with your code
    |
Step 6: Railway deploys
├─ Stops the old container (your old code)
├─ Starts the new container (your new code)
└─ This takes ~30-60 seconds
    |
Step 7: Your website is updated
└─ Open https://government-assets-platform-production...
    → You see your changes

```

Total time from `git push` to website update: ~1-2 minutes.

Q11: "What happens to the database when I push new code?"

Nothing. Pushing code only updates the application code (Django views, templates, CSS). The database keeps all its data.

Operation	Database affected?
Change a color in CSS	No — only frontend code changed
Change a template (HTML)	No — only frontend code changed
Add a new feature (new view)	No — new code only
Add a new database field to a model	Yes — you need to run migrations
Delete a model	Yes — you need to run migrations

When you add a new field or model:

1. You edit `models.py` locally
 2. You run `python manage.py makemigrations` (creates migration file)
 3. You run `python manage.py migrate` locally (applies to local DB)
 4. You commit the new migration file along with your code changes
 5. Push to GitHub
 6. Railway will run `python manage.py migrate` automatically during deployment
 7. The remote database gets the new field/table
-

Q12: "I keep hearing about Docker — what IS it really?"

Docker is a tool that packages an application with everything it needs to run, so it works the same on any computer.

Without Docker:

"Works on my machine!" – The classic developer excuse.
Your app works on your laptop but fails on the server because:

- Different operating system
- Different Python version
- Missing system libraries
- Different database version

With Docker:

Docker packages:

```
Your Application
+ Python 3.13
+ All pip packages
+ System dependencies
+ Configuration files
= DOCKER CONTAINER
```

This container runs identically on:

- Your laptop ✓
- Railway ✓
- Any other server ✓

Why Railway uses Docker:

- Railway doesn't know what your laptop looks like
- Docker guarantees the container runs the same way every time
- If the container crashes, Railway restarts it
- Railway can run many containers without them interfering with each other

On your laptop, you DON'T use Docker because:

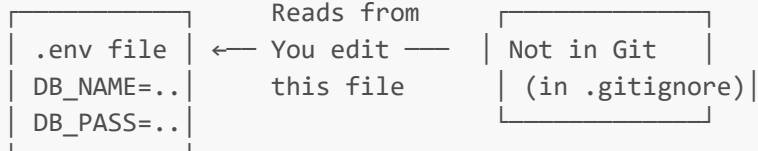
- Django's `runserver` works fine directly on Windows
- Keycloak's `kc.bat start-dev` works fine directly on Windows
- Adding Docker locally would add complexity without benefit
- Docker Desktop requires a paid license for commercial use

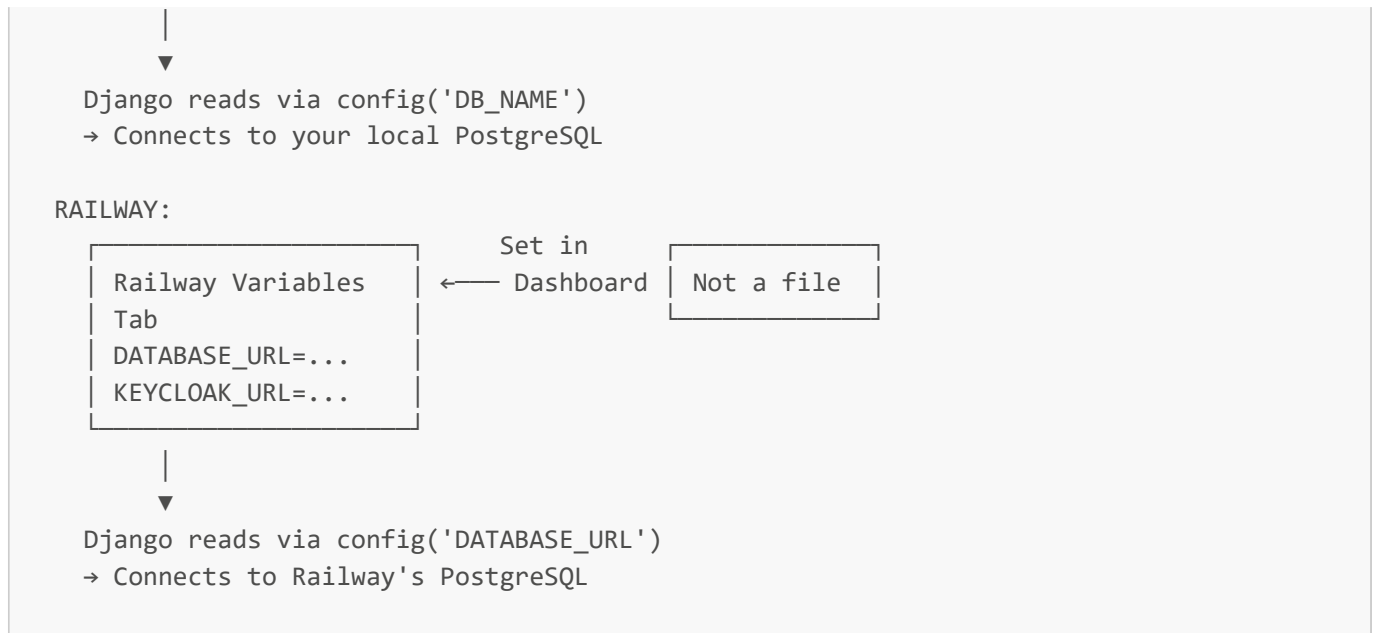
Q13: "What about the `.env` file? Is it on Railway too?"

.env file exists ONLY on your laptop. It is NOT uploaded to GitHub and NOT used on Railway.

How environment variables work in each place:

YOUR LAPTOP:





Key point: The `.env` file is listed in `.gitignore` — it is NEVER pushed to GitHub. This is a security measure so your passwords don't end up in public source code.

On Railway: You manually type the same values into the Variables Tab in the Railway Dashboard.

Q14: "If someone asks me to explain the whole system in 2 minutes, what do I say?"

The elevator pitch:

"Our system has three main parts. **Django** serves the website — it handles user requests, displays pages, and enforces permissions. **PostgreSQL** stores all the data — users, assets, audit logs — with each ministry getting its own isolated schema for security. **Keycloak** handles authentication — users log in through Keycloak, which verifies their identity and passes a secure token to Django.

The whole system is hosted on **Railway**, which provides free hosting with automatic HTTPS and PostgreSQL. Every time I push code to GitHub, Railway automatically rebuilds and redeploys the site — that's continuous deployment.

For development, everything runs on my laptop without Docker — Django through `runserver`, Keycloak through its built-in command, and PostgreSQL as a Windows service. The code is designed to work in both environments by reading configuration from environment variables — `.env` file locally, Railway Variables tab online."

Summary of Key Insights for Your Presentation

If judge asks...	Your answer should include...
"How does multi-tenancy work?"	"Each ministry gets a separate PostgreSQL schema. Django switches schemas using <code>search_path</code> . Data is isolated at the database level."
"How does authentication work?"	"Keycloak handles passwords and identity. Django handles roles and permissions. The two systems communicate via signed JWT tokens."

If judge asks...	Your answer should include...
"How is the project hosted?"	"Django and Keycloak run on Railway as separate containers. They share a PostgreSQL database. Railway provides HTTPS automatically."
"How do you deploy updates?"	"We push code to GitHub. Railway detects the push, rebuilds the Docker container, and redeploys. The whole process takes about a minute."
"What happens if Railway goes down?"	"Currently, our service goes down. For production, we would add redundancy — multiple servers, load balancers, and a backup provider."
"How would you scale this?"	"Add more Gunicorn workers. Add Redis caching. Move to a dedicated PostgreSQL. Use a CDN for static files. Deploy multiple Django instances behind a load balancer."
"How is security ensured?"	"Five layers: HTTPS (transport), Keycloak (authentication), Django roles (authorization), PostgreSQL schemas (data isolation), and audit logs (accountability)."